RAG Pipeline Step 2: Retrieval

1. Definition

Retrieval is the "R" in RAG. It is the real-time process of fetching the most relevant information (context) from the indexed database in response to a user's query. The goal of this step is *not* to answer the question, but to find the raw text chunks that are most likely to *contain* the answer.

2. Core Process (Vector Search)

The most common retrieval method works as follows:

1. **Embed the Query:** The user's query (e.g., "What is the RAG policy?") is converted into a vector using the *same embedding model* that was used during the Indexing step.
2. **Similarity Search:** The system compares this query vector to all the vectors in the database to find the "closest" matches (the "top-k" results).
3. **Return Context:** The retriever returns the corresponding text chunks (as Document objects) for these top-k vectors. This context is then "augmented" into the prompt for the next step (Generation).

3. Tools & Frameworks

In LangChain, retrieval is abstracted by the BaseRetriever interface. This allows for many different strategies beyond simple vector search.

Tool / Strategy	Explanation (in LangChain)	Reason for Use
VectorStoreRetriever	The most common retriever. It is created from a vector store (e.g., vectorstore.as_retriever()) and uses similarity search.	This is the standard for performing semantic search and is the backbone of most RAG applications.
Maximum Marginal Relevance (MMR)	A search mode for the VectorStoreRetriever (search_type="mmr").	It finds documents that are *both* relevant to the query *and* different from each other. This **increases diversity** in the results and avoids retrieving 5 chunks that all say the same thing.

MultiQueryRetriever	This is a "Query Translation" technique. It uses an LLM to generate *multiple* versions of the user's query, retrieves documents for all of them, and combines the results.	**Overcomes query wording issues.** If the user's phrasing doesn't match the documents, the LLM's generated queries might. This is a form of RAG Fusion.
SelfQueryRetriever	This is an advanced "Query Construction" tool. It uses an LLM to parse the user's query (e.g., "Find me emails from John from last week") into two parts: a semantic query ("emails") and metadata filters (author="John", date > ...).	**Enables structured search.** It allows users to use natural language to query based on metadata, which is impossible with standard semantic search.
ContextualCompressionRetriever	This is a "Re-ranking" / "Filtering" step. It wraps a base retriever. After retrieving full documents, it uses an LLM to "compress" them, extracting *only* the specific sentences that are relevant to the query.	**Reduces noise.** It passes a smaller, more focused context to the final LLM, which can improve answer quality and reduce cost.